

Developer Documentation & Codebase Tour

Project: LearnLens AI

1. Directory Structure

```
c:\Users\HP\OneDrive\Documents\Soham\Projects\Transcript Agent\  
■■■ backend/  
■ ■■■ app/  
■ ■ ■■■ api/  
■ ■ ■ ■■■ endpoints.py # REST endpoints (sessions, frames, chat, artifacts)  
■ ■ ■ ■■■ websocket.py # Real-time WebSocket streaming (transcripts, frames)  
■ ■ ■■■ core/  
■ ■ ■ ■■■ config.py # App configurations, thresholds, and paths  
■ ■ ■■■ db/  
■ ■ ■ ■■■ database.py # SQLite database initialization and CRUD managers  
■ ■ ■■■ models/  
■ ■ ■ ■■■ schemas.py # Pydantic schemas and domain models  
■ ■ ■■■ providers/ # Model abstraction layer  
■ ■ ■ ■■■ base.py # Abstract Base Classes (LLM, VLM, STT, OCR, Embedding)  
■ ■ ■ ■■■ llm_provider.py # Ollama LLM provider with fallback tutor engine  
■ ■ ■ ■■■ vlm_provider.py # Vision model provider with CV heuristic analysis  
■ ■ ■ ■■■ transcription_provider.py # faster-whisper integration  
■ ■ ■ ■■■ ocr_provider.py # PyTesseract / text extraction  
■ ■ ■ ■■■ embedding_provider.py # Subword TF-IDF cosine embedding space  
■ ■ ■■■ services/ # Core business intelligence engines  
■ ■ ■ ■■■ capture_service.py  
■ ■ ■ ■■■ screenshot_service.py # dHash deduplication and frame processing  
■ ■ ■ ■■■ knowledge_service.py # Concept, definition, and evidence extraction  
■ ■ ■ ■■■ rag_service.py # Session-scoped vector + keyword retriever  
■ ■ ■ ■■■ teacher_service.py # AI Teacher with 12 pedagogical modes  
■ ■ ■ ■■■ task_agent_service.py # Human control state machine & command runner  
■ ■ ■ ■■■ ppt_service.py # python-pptx presentation deck generator  
■ ■ ■ ■■■ pdf_service.py # ReportLab dual PDF generator (Visual & Report)  
■ ■ ■ ■■■ quiz_service.py # PyMuPDF quiz importer & gap diagnostic engine  
■ ■ ■ ■■■ demo_service.py # High-yield synthetic educational session seeder  
■ ■ ■ ■■■ video_import_service.py # Offline local video processor (MP4/WebM)  
■ ■ ■■■ main.py # FastAPI application entry point & lifespan  
■ ■■■ tests/ # Automated pytest test suites  
■ ■■■ requirements.txt  
■■■ frontend/  
■ ■■■ src/  
■ ■ ■■■ components/ # Modular React components  
■ ■ ■ ■■■ Sidebar.tsx  
■ ■ ■ ■■■ HumanControlBar.tsx  
■ ■ ■ ■■■ ChatInterface.tsx  
■ ■ ■ ■■■ SessionDashboard.tsx  
■ ■ ■ ■■■ LiveCaptureStudio.tsx  
■ ■ ■ ■■■ SlideCollection.tsx  
■ ■ ■ ■■■ TranscriptViewer.tsx  
■ ■ ■ ■■■ StudyArtifacts.tsx  
■ ■ ■ ■■■ QuizPracticeModal.tsx  
■ ■ ■ ■■■ LearnerProfileView.tsx  
■ ■ ■ ■■■ StartLearningModal.tsx  
■ ■ ■■■ services/ # Frontend services (api.ts, websocket.ts, mediaCapture.ts)  
■ ■ ■■■ types/index.ts # TypeScript interface definitions  
■ ■ ■■■ App.tsx # Root application coordinator  
■ ■ ■■■ index.css # Tailwind CSS & sleek scrollbars  
■ ■■■ package.json
```

```
■ ■■■ vite.config.ts
■■■ extension/ # Manifest V3 Chrome Extension source
■■■ data/ # Persistent local SQLite DB & session files
■■■ docs/ # Comprehensive technical documentation & PDFs
■■■ run.py # One-command master launcher
■■■ start_learnLens.bat # Windows 1-click batch launcher
■■■ start_learnLens.sh # Unix 1-click shell launcher
```

2. Implementing New Model Providers

All model providers inherit from the abstract base classes in ``backend/app/providers/base.py``:

- ``LLMProvider``: Implement ``generate_response(prompt, system_prompt, kwargs)`` and ``is_available()``.
- ``VLMPProvider``: Implement ``analyze_image(image_path, prompt)`` and ``is_available()``.
- ``TranscriptionProvider``: Implement ``transcribe_audio_file(audio_path)``.
- ``EmbeddingProvider``: Implement ``get_embedding(text)``.
- ``OCRProvider``: Implement ``extract_text(image_path)``.

Because business logic references the abstract interfaces rather than concrete vendor SDKs, you can swap between local Ollama, vLLM, llama.cpp, or cloud providers with zero refactoring in the core services.